

ANSIBLE INTERVIEW CHEAT SHEET

Keywords & technical terms only — quick-scan before the interview

CORE & ARCHITECTURE

Ansible: RedHat IaC/config mgmt, agentless, push-based, declarative YAML

Control node (ansible installed) → SSH/WinRM → Managed nodes (Python only, no agent)

Module exec: control node ships small module script → runs on node → returns JSON → deleted

Contrast: Puppet/Chef = agent-based, pull model, Ruby/DSL

EXECUTION FLOW

Flow: Inventory → Playbook parsed → Facts gathered → Tasks run per host → Handlers fire → Report

Idempotency: same run N times = same end state, no side effects (core concept)

INVENTORY

Static: INI/YAML file, groups [webservers]

Dynamic: plugin-generated (AWS EC2/Azure/GCP), aws_ec2.yml, keyed_groups

Vars: group_vars/, host_vars/ dirs

Nested groups: :children syntax

Check: ansible-inventory -i inv --list

PLAYBOOK STRUCTURE

Hierarchy: Playbook → Play → Tasks → Modules

Play keys: hosts, become, vars, tasks, handlers, roles

Task: name, module, register, when, loop, tags, notify

MODULES · COMMAND VS SHELL

Key modules: package, file, copy, template, service/systemd, user, git, uri, debug

copy vs template: copy = static file as-is; template = Jinja2-rendered dynamic file

command: no shell features (pipes/redirects), safer, no shell injection — PREFER

shell: runs via /bin/sh, supports | > && env vars — use only when needed

CLI COMMANDS

ansible all -m ping — connectivity test

ansible-playbook site.yml run playbook

--check --diff dry-run + show file diffs

--tags / --skip-tags selective run

-e 'k=v' extra-vars -i inv inventory

--ask-become-pass / -K sudo pw prompt

ansible-doc <module> module docs offline

ansible-lint · ansible-galaxy

VARIABLES & PRECEDENCE

Low→High: role defaults < inventory vars < playbook vars <

role vars < include/block/task vars < facts/registered <

extra-vars (-e) — ALWAYS wins, highest

Types: string, number, bool, list, dict; default() filter

FACTS

Facts: auto-collected host info (OS, IP, mem, CPU) via setup module

ansible_facts['os_family'], ansible_facts['hostname']

Perf: gather_facts: false to skip when unneeded; fact caching for speed

HANDLERS & NOTIFY

Handlers: run only when notify'd by a changed task (e.g. restart svc after config change)

Note: run once at END of play even if notified multiple times; meta: flush_handlers to force earlier

LOOPS & CONDITIONALS

loop: loop: [a,b,c] replaces deprecated with_items

Complex loop: loop: dict | dict2items for dict iteration

when: when: ansible_os_family == 'Debian' — conditional exec

Combine: when: list = AND; use or/and explicitly for complex logic

TAGS

Tags: label tasks/roles/plays; --tags install, --skip-tags slow

Special: always tag runs every time; never needs explicit call

ROLES & GALAXY

roles/x/{tasks,handlers,templates,defaults,vars,files,meta}/main.yml

defaults vs vars: defaults = low priority/overridable; vars = high priority/locked

meta/main.yml: role dependencies (run first automatically)

ansible-galaxy role init scaffold · ansible-galaxy install <role> reuse community role

TEMPLATES · JINJA2

Syntax: {{ var }} interpolation, {% if %}{% for %} blocks

Filters: default(), upper, join, basename

lookup(): lookup('env','VAR'), lookup('file', path)

VAULT & SECRETS (incl. AWS)

ansible-vault encrypt/decrypt/view/edit <file>

ansible-vault encrypt_string 'x' --name key

Run: --ask-vault-pass or vault_password_file in cfg

Multi-vault: --vault-id prod@.vault_prod per-env passwords

AWS Secrets Mgr integ: install amazon.aws collection, then:

lookup('amazon.aws.aws_secret',
'prod/db/pass', region='ap-south-1')

→ avoids hardcoded/file-based secrets in prod, pulled at runtime

PRIVILEGE ESCALATION

become: become: true, become_user: root, become_method: sudo

Override: set become: false at task level to drop escalation

--ask-become-pass / -K interactive sudo prompt

COLLECTIONS

Collections: namespaced bundles of modules/roles/plugins (modern distribution unit)

ansible-galaxy collection install amazon.aws

Usage: FQCN e.g. amazon.aws.ec2_instance, or declare collections: at play level

ANSIBLE CONFIGURATION (ansible.cfg)

Precedence: ENV \$ANSIBLE_CONFIG > ./ansible.cfg > ~/.ansible.cfg > /etc/ansible/ansible.cfg

[defaults] inventory, remote_user, forks,
gathering, fact_caching, vault_password_file
[privilege_escalation] become, become_method
[ssh_connection] pipelining=True, control_path

ERROR HANDLING

ignore_errors: continue play despite task failure

failed_when / changed_when: custom failure/change logic override

block: / rescue: / always: — try/catch/finally

Batch safety: serial + max_fail_percentage for rolling runs

DYNAMIC INVENTORY (AWS)

plugin: amazon.aws.aws_ec2

regions: [ap-south-1]

filters: {tag:Env: production}

keyed_groups: [{key: tags.Role}]

PERFORMANCE TUNING

Speed up: pipelining=True, forks=20, gathering=smart, fact caching, async tasks

delegate_to: run task on different host (e.g. deregister from LB)

local_action: shorthand to run task on control node

TESTING & VALIDATION

ansible-playbook --syntax-check

ansible-lint --check --diff dry-run

Molecule: test roles in Docker before prod (CI gate)

CI/CD INTEGRATION

lint → syntax-check → --check (dry-run PR gate) → apply on merge

Secrets in CI: vault_password_file or CI secret store, never committed

Terraform+Ansible: TF provisions infra → Ansible configures it (same pipeline)

PRODUCTION PATTERNS

Rolling deploy: serial: 1 or 25% + uri health-check per batch

Blue/green: deploy to idle env, validate, switch LB target group

Tower/AWX: Web UI, RBAC, scheduling, credential vault, audit logs, job templates

GitOps: desired state in Git; playbooks reconcile drift

COMMON ERRORS

Host key error: host_key_checking=False in cfg (non-prod) or accept key

Perm denied: missing become: true or wrong SSH user

Not idempotent: raw shell/command without creates/changed_when

Fact gather slow: gather_facts: false if not needed

BEST PRACTICES

small reusable roles; vault/secrets manager for secrets always

idempotent tasks; avoid raw shell where a module exists

pin collection versions; review --check --diff before prod apply

consistent tagging & naming; dynamic inventory over static in cloud